COMPUTER SOCIETY OF INDIA(CSI)
COEP TECH STUDENT CHAPTER

with

Simplify Healthcare

presents

INSPIRON 5.0

A NATIONAL LEVEL HACKATHON

in association with

INFOTECH WORKS

ClaimCraft

By - **Cipher Crew**

Sr.No.	Name	GitHub
1	Akshit Agarwal	/Akshit926
2	Vitthal Shahaji Humbe	/vitthalhumbe
3	Avinash Dharmaraj Pawar	/Avinash14-coder
4	Dhruv Vandana Vaswani	/Dhruv1309

1.Problem Statement

The United States healthcare system processes hundreds of millions of insurance claims annually. Each claim travels from a provider to a payer as a structured X12 Electronic Data Interchange (EDI) file. When that file contains an error — even a single incorrect character — the entire claim is rejected. Healthcare providers currently lose an estimated $262 billion annually to claim rejections, and 86 percent of those denials are caused not by clinical disputes but by preventable EDI format errors. A single transposed digit, a missing decimal separator, or a misconfigured Place of Service code can hold a $150,000 claim for up to 30 days, with no plain-English explanation of what went wrong or how to fix it.

The root cause is a structural timing failure. Payer validation systems are sophisticated, but they apply their checks after the file has been transmitted and queued — a process that takes 14 to 30 days. By the time a provider receives a terse rejection code with no explanation, the same misconfiguration may have been replicated across hundreds of subsequent claims. Clearinghouses — the intermediaries that convert practice management data into EDI — perform only basic structural checks, confirming that a file is well-formed X12, but not that the codes inside it are semantically valid or clinically consistent. This gap between structural correctness and semantic validity is where ClaimCraft intervenes.

2.Abstract

ClaimCraft is a full-stack X12 EDI parsing, validation, and AI-assisted correction platform built to address a structurally broken process in US healthcare billing. The system accepts raw EDI files in the 837P, 837I, 835, and 834 transaction formats, converts them into human-readable structured representations, validates them against over 400 encoded HIPAA 5010 compliance rules, and provides an AI-powered conversational assistant that answers contextual questions about the file and its errors in plain English — all in under 200 milliseconds.

The platform is built on a pure Python parser pipeline with a FastAPI backend and a React + TypeScript frontend. Validation is performed in two tiers: a static tier that evaluates format, structure, qualifier codes, and cross-segment consistency without any external calls, and a full validation tier that performs live lookups against the CMS NPPES NPI Registry and the NLM ICD-10-CM API. The AI assistant uses a Groq-hosted LLaMA 3.3 70B model, injected with the full parsed file as context, to provide grounded answers about any segment, element, or error in the uploaded file. Additional capabilities include 835 vs 837 payment reconciliation, batch ZIP processing of mixed-type EDI files, and a 834 member change delta report.

ClaimCraft was developed by Team Cipher Crew from Pimpri Chinchwad College of Engineering, Pune, as a submission to INSPIRON 5.0 — a national-level hackathon organised by the Computer Society of India COEP Tech Student Chapter in association with Simplify Healthcare, for Problem Statement PS01: US Healthcare EDI Parser and X12 File Validator (837/835/834).

3.Proposed Solution

ClaimCraft moves the point of validation from the payer's queue to the provider's workstation. Rather than discovering an error 14 to 30 days after a file has been transmitted, a billing specialist uploads a raw EDI file and receives a complete picture of it in under 200 milliseconds: a human-readable loop tree, a plain-English error report with exact fix suggestions, and an AI assistant that can answer any question about the file grounded in its actual contents

3.1 Core Capabilities

The parser accepts .edi, .txt, and .zip files, automatically detects the transaction type from the ISA/GS/ST envelope, and assembles the complete nested loop hierarchy into a structured JSON tree — making the unreadable, readable. The validation engine walks that tree against over 400 encoded HIPAA 5010 rules, checking every required segment, element format, qualifier code, cross-segment dependency, and live NPI and ICD-10 registry — returning a precise, plain-English error list with exact segment location and suggested fix. The AI assistant receives the full parsed file and error list as context and answers any follow-up question about the file, including what any segment means, why an error was raised, and what the correct value should be.

3.2 Key Features

Feature	Description	Benefit
Auto Type Detection	Identifies 837P / 837I / 835 / 834 from ISA/GS/ST segments	No manual file classification needed
Segment Tree UI	Interactive collapsible view of every loop and element with human-readable names	Makes opaque EDI navigable
Validation Engine	400+ HIPAA 5010 rules: mandatory checks, NPI Luhn, ICD-10/CPT codes, cross-segment consistency	Catches errors before payer submission
AI Chat Assistant	LLM-powered chat answers contextual questions about the uploaded EDI file	Eliminates need for HIPAA manual lookups
835 vs 837 Reconcile	Side-by-side claim vs payment — reveals exactly why each claim was denied or adjusted	Identifies underpayments instantly
One-Click Auto-Fix	AI proposes corrected value; accept and the file re-validates immediately	Reduces correction time from hours to seconds
Batch ZIP Upload	Upload a ZIP of mixed EDI files; auto-detects and validates each in parallel	Handles production-scale billing batches
Real NPI Live Validation	Calls CMS NPPES Registry API to confirm NPI is active and name-matched	Catches provider identity errors before rejection

4. Methodology

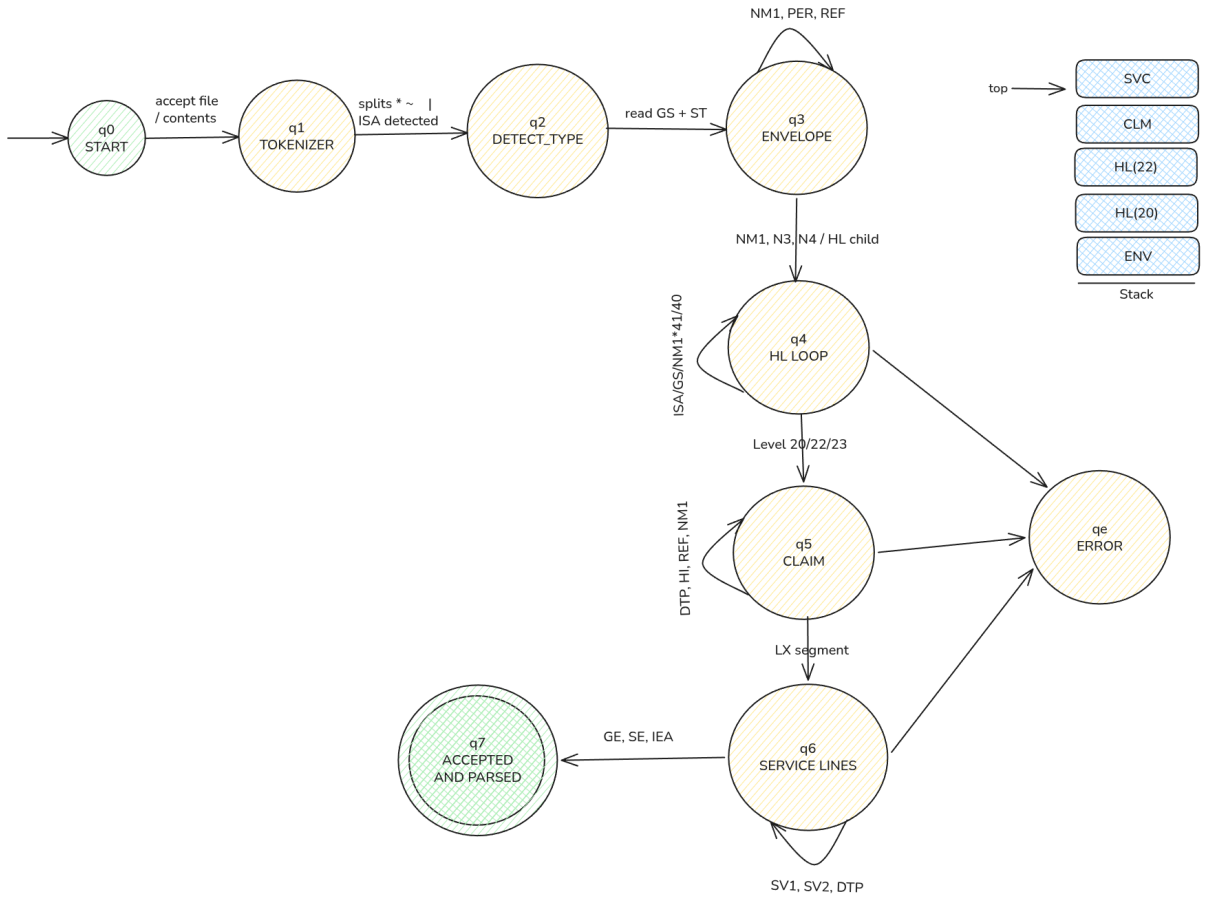
ClaimCraft is built around three independently designed but tightly coupled engines: the parser pipeline, the validation engine, and the AI assistant. Each engine is designed to be correct-first — producing accurate, auditable output — and fast-second, targeting sub-200-millisecond total response time for static validation.

4.1 The Parser Pipeline

The parser is written entirely in pure Python with no external parsing libraries. This gives complete control over how loops are identified, how ambiguous segment sequences are resolved, and how errors in the source file are handled gracefully. The pipeline operates in five sequential stages. First, delimiter detection reads the fixed 106-character ISA header to extract the element separator, component separator, and segment terminator. Second, tokenisation splits the raw text into segments and elements using those delimiters, normalising Windows line endings, UTF-8 BOM characters, and newline-separated segment terminators. Third, transaction classification routes the file to the appropriate parser module based on the ST02 transaction identifier and the GS08 implementation convention. Fourth, loop hierarchy construction assembles the complete nested loop tree using HL hierarchical level segments to determine parent-child relationships. Fifth, element mapping attaches human-readable names to every element position, transforming CLM01 into Claim Control Number and NM103 into Last or Organisation Name.

CORE LOGIC

1. 837P and 837I parser -



Parser Model: Pushdown Automaton (PDA)

The 837 parser is modeled as a Pushdown Automaton with 8 states and a segment stack. The stack tracks the current hierarchical context so the parser always knows which loop it is inside at any point during traversal.

q0 → START Initial state. The parser accepts the raw file content as a byte stream or string and passes it to the tokenizer.

q0 → q1 TOKENIZER The raw content is split on the element separator * and segment terminator ~, producing a flat list of segments. The ISA segment is identified first to extract delimiters. If ISA is not detected, the parser transitions to qe ERROR.

q1 → q2 DETECT_TYPE The ST segment is read. ST01 determines the transaction type: 837 routed to either the 837P or 837I handler based on ST03 implementation convention reference (005010X222A1 for Professional, 005010X223A2 for Institutional). Unknown ST01 values transition to qe ERROR.

q2 → q3 ENVELOPE GS and ST segments are consumed and their metadata (sender ID, receiver ID, version, group control number) is pushed onto the ENV stack frame. NM1*41 (submitter) and NM1*40 (receiver), PER, and REF segments are consumed here. The envelope frame stays on the stack for the lifetime of the parse.

q3 → q4 HL LOOP On encountering an HL segment, the parser reads HL03 (hierarchical level code) to determine loop type: 20 = Billing Provider, 22 = Subscriber, 23 = Patient. The corresponding HL frame is pushed onto the stack. NM1, N3, N4 (address), SBR, and REF segments are consumed within this state. If ISA, GS, or NM1*41/40 segments are seen, the parser pops back to ENVELOPE. The HL LOOP state is re-entered for each nested HL segment (levels 20 → 22 → 23).

q4 → q5 CLAIM On encountering a CLM segment, a CLM frame is pushed onto the stack. DTP (service dates), HI (diagnosis codes), REF (claim references), and NM1 segments for rendering provider, service facility, and payer are consumed here. An unexpected or out-of-sequence segment transitions to qe ERROR.

q5 → q6 SERVICE LINES On encountering an LX segment (line counter), an SVC frame is pushed. SV1 (professional) or SV2 (institutional) and DTP (service date) segments are consumed per line. Each LX pops the previous SVC frame and pushes a new one. The SERVICE LINES state loops on itself for each line.

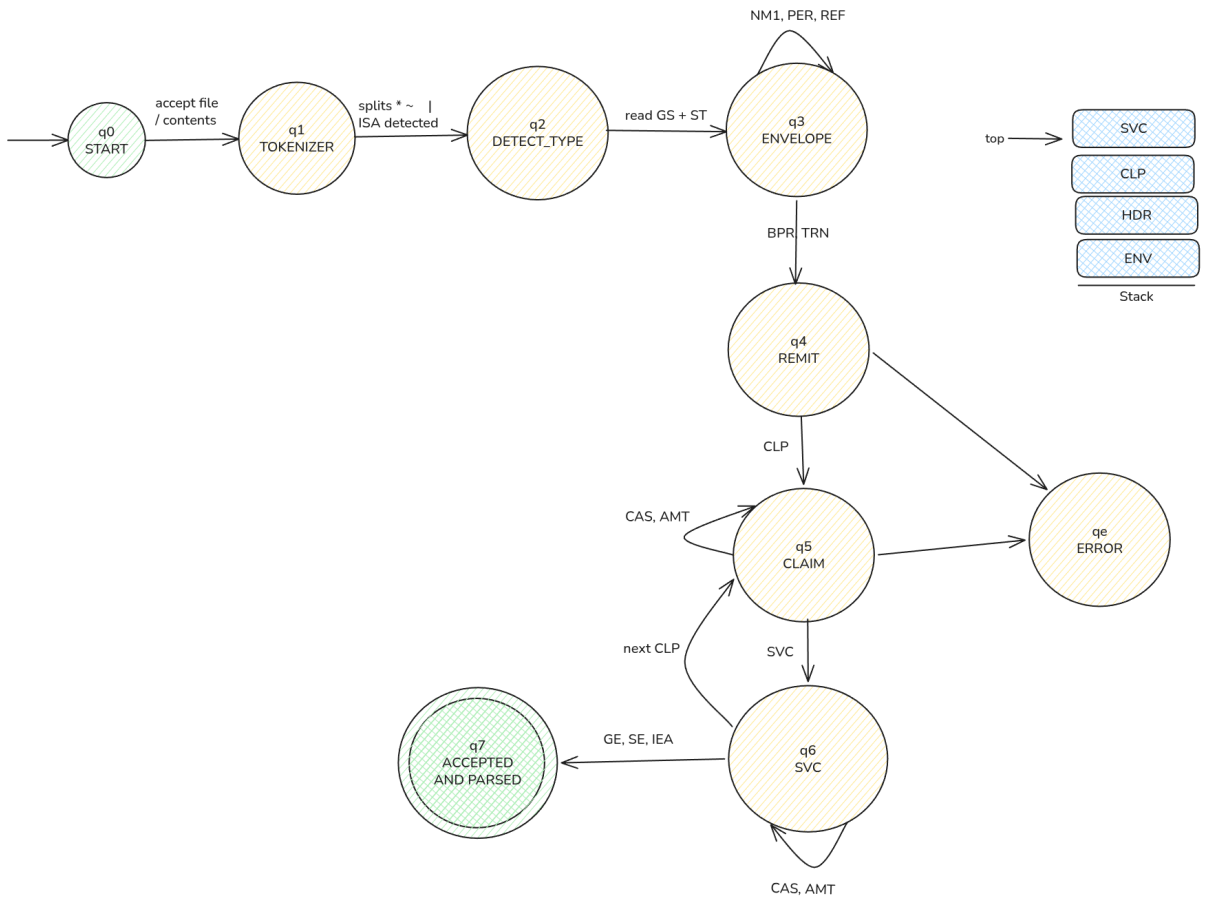
q6 → q7 ACCEPTED AND PARSED On encountering GE, SE, or IEA (trailer segments), all remaining stack frames are popped in order, the hierarchical JSON tree is finalised, and the parser transitions to the accept state. The output is a structured nested dict representing the complete loop hierarchy.

Any state → qe ERROR Malformed segment terminators, unrecognised ST codes, missing mandatory segments, or stack underflow transitions any state to ERROR. The parser does not halt — it logs the error with position and segment ID, attempts recovery by skipping the malformed segment, and continues from the current state.

Key Design Decisions

The stack-based model handles the recursive nature of HL loops without recursion in code — each HL segment is a push, each trailer is a pop. This means a single 837 file containing hundreds of claims with nested subscriber and patient loops is parsed in a single linear pass through the segment list, O(n) in the number of segments.

2. 835 parser :



The 835 parser is modeled as a Pushdown Automaton with 7 states and a segment stack. The stack tracks the current remittance context so the parser always knows whether it is inside a claim loop or service line at any point during traversal.

q0 → START Initial state. Accepts raw file content and passes it to the tokenizer.

q0 → q1 TOKENIZER Raw content is split on * and ~, producing a flat segment list. ISA is identified first to extract delimiters. If ISA is not detected, transitions to qe ERROR.

q1 → q2 DETECT_TYPE ST01 is read. Value 835 routes to the remittance handler. Unknown values transition to qe ERROR.

q2 → q3 ENVELOPE ISA, GS, ST segments are consumed and metadata pushed onto the ENV frame. NM1, PER, REF segments for payer and payee are consumed here. The envelope frame stays on the stack for the lifetime of the parse.

q3 → q4 REMIT On BPR and TRN segments, a HDR frame is pushed. Payment amount, trace number, and payment method are captured here.

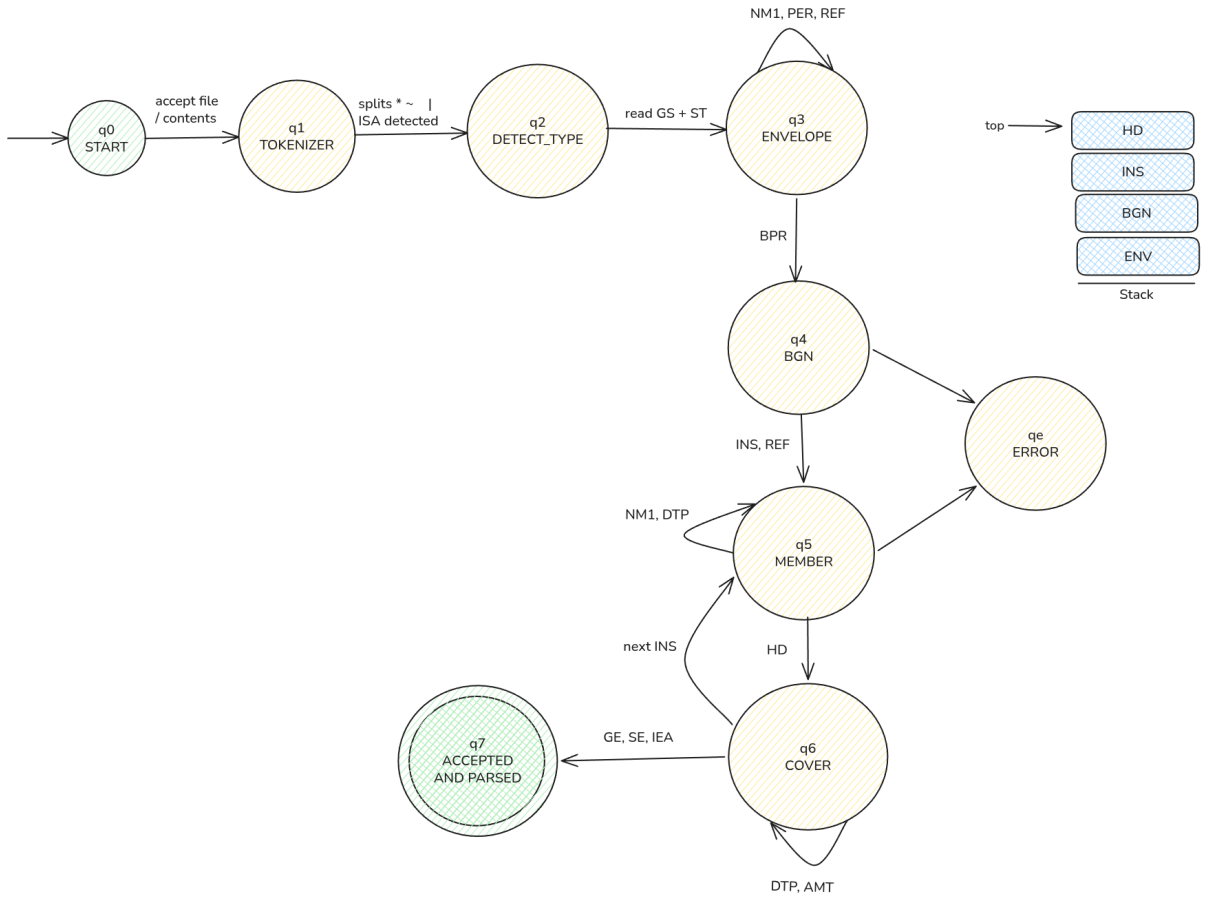
q4 → q5 CLAIM On CLP, a CLP frame is pushed. CAS adjustment groups and AMT segments are consumed here. On the next CLP segment, the current frame is popped and a new CLP frame is pushed — the state loops on itself for each claim.

q5 → q6 SVC On SVC, a SVC frame is pushed. Line-level CAS and AMT segments are consumed here. The SVC state loops on itself for each service line within the claim.

q6 → q7 ACCEPTED AND PARSED On GE, SE, or IEA, all remaining stack frames are popped, the remittance JSON tree is finalised, and the parser transitions to the accept state.

Any state → qe ERROR Malformed segments, unrecognised ST codes, or stack underflow transition to ERROR. The parser logs the error, skips the malformed segment, and continues.

3. 834 parser



The 834 parser is modeled as a Pushdown Automaton with 7 states and a segment stack. The stack tracks enrollment context so the parser always knows which member record and coverage block it is inside.

q0 → START Initial state. Accepts raw file content and passes it to the tokenizer.

q0 → q1 TOKENIZER Raw content is split on * and ~, producing a flat segment list. ISA is identified first to extract delimiters. If ISA is not detected, transitions to qe ERROR.

q1 → q2 DETECT_TYPE ST01 is read. Value 834 routes to the enrollment handler. Unknown values transition to qe ERROR.

q2 → q3 ENVELOPE ISA, GS, ST segments are consumed and metadata pushed onto the ENV frame. NM1, PER, REF segments for plan sponsor and insurer are consumed here. The envelope frame stays on the stack for the lifetime of the parse.

q3 → q4 BGN On BPR, a BGN frame is pushed capturing transaction purpose code, reference ID, and group number.

q4 → q5 MEMBER On INS and REF, a member frame is pushed. NM1 (member name), DMG (demographics), and DTP (effective and termination dates) are consumed here. On the next INS segment, the current member frame is popped and a new one pushed — the state loops on itself for each member record.

q5 → q6 COVER On HD, a coverage frame is pushed. DTP (coverage dates) and AMT segments are consumed here. The COVER state loops on itself for each coverage block within the member record.

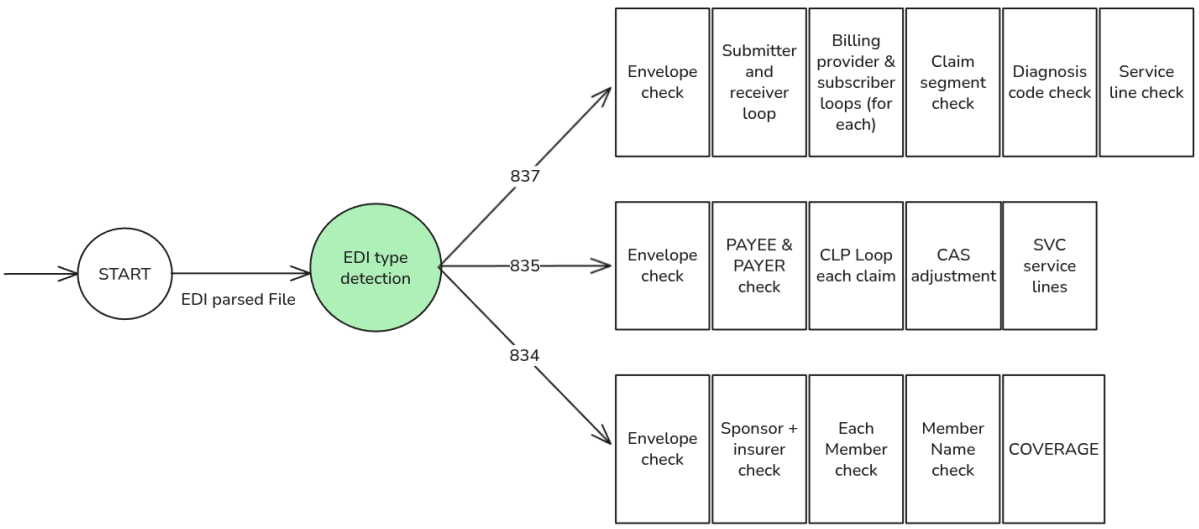
q6 → q7 ACCEPTED AND PARSED On GE, SE, or IEA, all remaining stack frames are popped, the enrollment JSON tree is finalised, and the parser transitions to the accept state.

Any state → qe ERROR Malformed segments, unrecognised ST codes, or stack underflow transition to ERROR. The parser logs the error, skips the malformed segment, and continues.

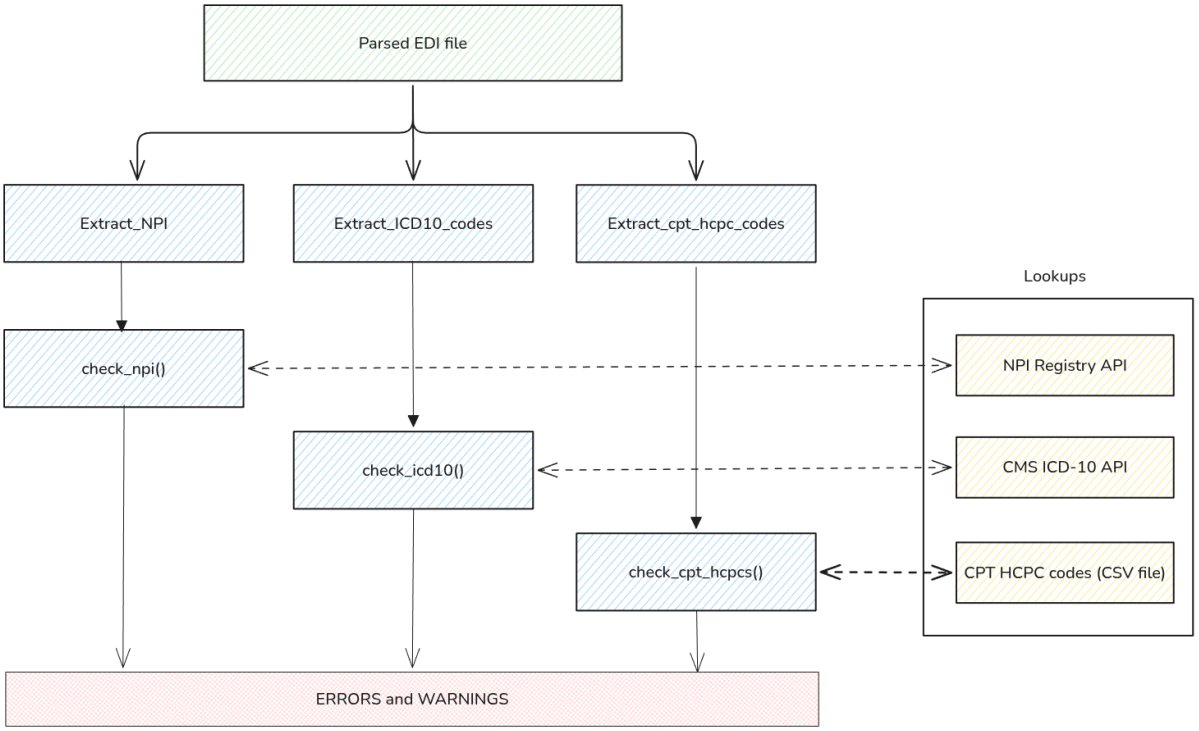
4.2 The Validation Engine

Validation operates in two tiers. Static validation evaluates the parsed JSON tree against 400+ encoded HIPAA 5010 rules without making any external API calls. This covers mandatory segment presence, element format correctness (amounts, dates, NPI format via Luhn algorithm), qualifier and code set validity (CMS Place of Service codes, ICD-10 format, CPT/HCPCS format), and cross-segment logical consistency (CLM02 total must equal the sum of all SV1 service line charges, service date must not precede date of birth, future service dates are flagged). Full validation adds live API calls to the CMS NPPES NPI Registry to confirm that each NPI is active and name-matched, and to the NLM ICD-10-CM API to confirm that each diagnosis code exists in the current year's active code set.

1. Static Validation Engine (Internal validation)



2. Live Validation Engine (CMS - lookup validation)



4.3 The AI Assistant

The AI assistant uses a Groq-hosted LLaMA 3.3 70B model configured at temperature 0.2 for deterministic, factual responses. On the first message of each session, the full parsed file — including the complete loop tree serialised as compact JSON and the full validation error list — is injected as structured context. The system prompt instructs the model to act as an expert X12 EDI healthcare billing consultant, to reference actual values from the file rather than generic descriptions, and to always state the exact wrong value and the exact correction when explaining an error. On subsequent messages, only a compact summary is prepended to prevent context window overflow during long conversations.

4.4 Technology Stack

Layer	Technology	Role
Parser	Python (scratch-built)	X12 tokeniser, loop hierarchy builder, element mapper

Validation Rules	JSON / YAML	400+ HIPAA 5010 rules encoded as data; extensible without code changes
AI Assistant	Groq API — LLaMA 3.3 70B	File-grounded Q&A with sub-second inference latency
Backend API	FastAPI (Python)	Async REST endpoints: parse, validate, chat, reconcile
Frontend	React + TypeScript + Vite	Three-panel claims UI, reconciliation view, batch view
Reference Data	CMS NPPES + NLM ICD-10 + HCPCS 2026	Live NPI lookup, diagnosis and procedure code validation
Deployment	Vercel (frontend) + Railway Docker (backend)	CDN-distributed frontend, containerised backend

5. Workflow

ClaimCraft is designed around three primary user workflows: single file validation, 835 vs 837 payment reconciliation, and batch processing. Each workflow follows the same underlying pipeline — upload, parse, validate — but surfaces different views and outputs depending on the task.

5.1 Single File Validation

The user uploads a raw .edi or .txt file. The frontend sends the file to POST /parse. The backend parses the file, runs static validation, and returns the loop tree and error list in under 200 milliseconds. The frontend navigates to the three-panel Claims view. The left panel displays the raw EDI text with segment identifiers highlighted and error lines marked in red. The centre panel shows the collapsible humanised loop tree with error fields indicated inline. The right panel lists plain-English validation errors with exact fix suggestions. The user may optionally click Full Validate to trigger live NPI and ICD-10 registry checks. The AI chat assistant is available throughout to answer any question about the file.

5.2 835 vs 837 Reconciliation

The user uploads the original 837 claim file and its corresponding 835 remittance advice. Each file is parsed independently via POST /parse. The frontend then sends both parsed objects to POST /reconcile. The engine matches claims by CLM01 in the 837 against CLP01

in the 835, computing the billed amount, paid amount, patient responsibility, delta, and all CAS adjustment reasons with plain-English CARC code descriptions for each matched pair. The results are displayed as a summary strip and a per-claim results table, instantly revealing underpayments, denials, and contractual adjustments.

5.3 Batch Processing

The user uploads a single ZIP archive containing multiple EDI files of any supported transaction type. The backend extracts, auto-detects the type of each file, parses, and validates them in parallel. Results are returned as a consolidated report showing each file's detected type, segment count, error count, and validation status. Per-file parsed JSON is available for export.

5.4 System Data Flow

Step	Action
1. File Upload	User drops .edi / .txt / .zip onto the upload zone
2. Pre-processing	Backend sanitises delimiters, normalises encodings, detects file type
3. Type Detection	ISA → GS → ST scan identifies transaction type (837P / 837I / 835 / 834)
4. Parser Engine	Builds complete nested loop hierarchy into structured JSON tree
5. Validation Engine	Runs 400+ HIPAA 5010 static rules; optionally runs live API checks
6. Three Output Panels	Segment Tree UI · Error Report with one-click fix · 835/834 Summary
7. AI Assistant	Gemini / LLaMA model receives parsed JSON + error list as context
8. Export	Corrected EDI · Parsed JSON · Error PDF · 834 CSV

6. System Architecture

ClaimCraft follows a clean separation of concerns across three layers: the React + TypeScript frontend served via Vercel CDN, the FastAPI Python backend served via Railway Docker, and a set of external reference APIs consumed during full validation. All communication between the frontend and backend is via REST over HTTP; the frontend never communicates directly with external APIs.

6.1 API Endpoints

Route	Method	Purpose

/parse	POST	Parse EDI + run static validation. Accepts .edi .txt .zip
/validate	POST	Re-run static HIPAA 5010 validation on already-parsed data
/validate/full	POST	Static + live NPI, ICD-10, CPT API validation
/chat	POST	LLaMA AI chat with full parsed file injected as context
/reconcile	POST	Match 837 claim against 835 remittance by claim ID
/export	POST	Export parsed data — scaffolded, returns 501
/health	GET	Backend health check — returns { status: ok }

6.2 Backend Module Structure

The backend is organised into a core edi_parser package and a routes package. The edi_parser package contains core.py (tokeniser and delimiter detection), main.py (parse orchestrator), validator_static.py (400+ HIPAA rules), validator_live.py (NPI / ICD-10 / CPT API checks), reconciler.py (835 vs 837 matching logic), and a parsers/ directory containing the 837, 835, and 834 specific parsers. The routes package exposes the FastAPI path operations and handles request/response serialisation.

6.3 Frontend Panel Layout

The Claims view uses a fixed three-panel layout that fills the full viewport height. The left panel (280px, dark background) renders the raw EDI viewer with JetBrains Mono font, segment identifiers highlighted in light blue, element values in light green, and error lines given a red background with a 2px left border. The centre panel (flex-1, white) renders the collapsible humanised loop tree. The right panel (300px, white) renders the validation error cards and AI chat interface. Each panel scrolls independently within its own overflow container.

7. Validation Engine

The validation engine is the analytical core of ClaimCraft. It operates in two independently invokable tiers: static validation that completes in milliseconds without any external calls, and full validation that adds live API lookups for semantic correctness.

7.1 Static Validation Rules

Rule Category	What Is Checked
Mandatory Segments	Every required segment must be present in every required loop. Missing CLM in Loop 2300 is flagged immediately.
Element Formats	Amounts (decimal, no currency symbols), dates (YYYYMMDD, not in future), NPI (10 digits, Luhn checksum), identifier codes (correct format for qualifier type)
Qualifier & Code Sets	CLM05-1 against CMS Place of Service list, ICD-10 format validation, CPT/HCPCS format validation
Cross-Segment Consistency	CLM02 total must equal sum of all SV1 charges; service date must not precede date of birth; future service dates flagged; rendering provider NPI differences require PRV segment

7.2 Full Validation (Live API)

Check	API / Source
NPI Validation	CMS NPPES Registry API — confirms NPI is active and provider name matches registry
ICD-10 Codes	NLM ICD-10-CM API — confirms diagnosis code exists in current year's active code set
HCPCS/CPT Codes	Local HCPC2026_APR_ANWEB.txt file + NLM API fallback — confirms procedure code is valid

7.3 Error Output Format

Every error returned by the validation engine carries a machine-readable code, severity level, segment identifier, element reference, loop location, line number, plain-English message, and a specific suggested fix. This structured format allows the frontend to map each error directly to the correct line in the raw EDI viewer, the correct node in the segment tree, and the correct card in the error report panel.

8. AI Chat Assistant

The AI chat assistant transforms ClaimCraft from a validation tool into an interactive billing consultant. Rather than presenting errors as opaque codes and expecting the billing team to look them up in a 400-page HIPAA manual, ClaimCraft provides a conversational interface where users can ask any question about their file and receive answers grounded in its actual contents.

8.1 Model Configuration

Parameter	Value
Provider	Groq API
Model	LLaMA 3.3 70B Versatile
Temperature	0.2 — produces factual, deterministic analytical responses
Max Tokens	1024 per response
Context Strategy	Full parsed JSON + error list injected on first message; compact summary on subsequent messages

8.2 What the Assistant Can Do

The assistant can explain what any segment, element, or loop means using the actual values in the uploaded file. It describes why each validation error is raised, states the exact wrong value, and gives the precise correction. It interprets CARC and RARC codes in the context of reconciliation results. It can confirm whether a specific NPI, ICD-10, or CPT code is valid, explain which HIPAA 5010 rule governs a particular failure, answer general EDI terminology questions, and guide users through the correction process step by step.

9. Add-On & Bonus Features

9.1 835 vs 837 Reconciliation

Given an 837 claim file and its corresponding 835 remittance, the reconciliation engine matches by ICN/DCN claim identifier and produces a side-by-side breakdown of billed amount, paid amount, and all adjustment reasons. This instantly reveals underpayments, denials, and contractual adjustments without manual cross-referencing.

9.2 Batch ZIP Upload

A single ZIP archive containing multiple EDI files of mixed transaction types can be uploaded in one operation. The backend auto-detects each file's type, validates them in parallel, and returns a consolidated report across all files along with per-file JSON export.

9.3 834 Change Delta Report

Two 834 benefit enrollment files from consecutive months can be uploaded and auto-diffed. The engine identifies new additions, terminations, and attribute changes per member, catching errors before they cause coverage failures.

9.4 Real NPI Live Validation

During full validation, ClaimCraft calls the CMS NPPES NPI Registry API to confirm that each NPI in the file is active and that the provider name in the file matches the registry. Results are cached within the session to avoid redundant API calls during batch processing.

10. Potential Impact

Claim denials cost the US healthcare system $262 billion annually. 86 percent of those denials are preventable, and most are caused by fixable EDI format errors that ClaimCraft catches in under 200 milliseconds. A mid-size hospital reducing its rejection rate by just 5 percent saves over $50,000 annually in resubmission overhead and delayed revenue.

Target User	Use Case	ClaimCraft Benefit
Medical Billing Teams	Upload 837P before payer submission	Catch errors instantly instead of waiting 14–30 days
Health Insurance Payers	Validate inbound 835/834 files	Eliminate enrollment errors before they cause coverage gaps
Health-Tech Developers	Integrate EDI validation via API	No parser build required — use REST endpoints directly
Healthcare Institutions	Process monthly 834 batches	Auto-detect additions, terminations, and attribute changes

11. Running the Application

11.1 Prerequisites

Requirement	Version / Note
Python	3.12 or higher
Node.js	18 or higher
npm	9 or higher
Groq API Key	Free tier sufficient — for AI chat endpoint
Internet access	Required for /validate/full (CMS NPPES + NLM ICD-10 API calls)

11.2 Backend Setup

Navigate to the backend directory, create and activate a Python virtual environment, install dependencies from requirements.txt, and start the server with uvicorn app:app --reload --port 8000. Create a .env file inside the backend directory containing GROQ_API_KEY=your_key_here. The backend will be available at http://localhost:8000 with interactive API docs at http://localhost:8000/docs.

11.3 Frontend Setup

Navigate to the frontend directory, run npm install to install dependencies, and start the development server with npm run dev. Create a .env file inside the frontend directory containing VITE_API_BASE=http://localhost:8000. The frontend will be available at http://localhost:5173.

11.4 Common Issues

Symptom	Cause	Fix
Frontend shows Backend offline	Backend not started or wrong port	Run uvicorn on port 8000; check VITE_API_BASE
POST /chat returns 503	GROQ_API_KEY missing from .env	Add key to backend .env and restart uvicorn
CORS error in browser	Frontend URL not in allowed origins	Add http://localhost:5173 to allow_origins in app.py
POST /parse returns 400	Unsupported file format	Only .edi .txt .zip files are accepted
npm install fails	Node.js version too old	Upgrade to Node.js 18 or higher